РАСПРЕДЕЛЕННЫЕ И ОБЛАЧНЫЕ ВЫЧИСЛЕНИЯ

# Трехзвенная архитектура «клиент-сервер»

Клиент, сервер приложений и сервер базы данных: как устроены три уровня и зачем они нужны




Фаттахов Д. Р., группа ИДО ЗБ ПОАС-24-3 СБ, УрГЭУ

# План презентации

1. Клиент-сервер: основная идея
2. Двухзвенная модель и ее пределы
3. Три уровня: клиент, приложение, данные
4. Путь запроса через систему
5. Сравнение, плюсы и минусы
6. Технологии, пример, масштабирование

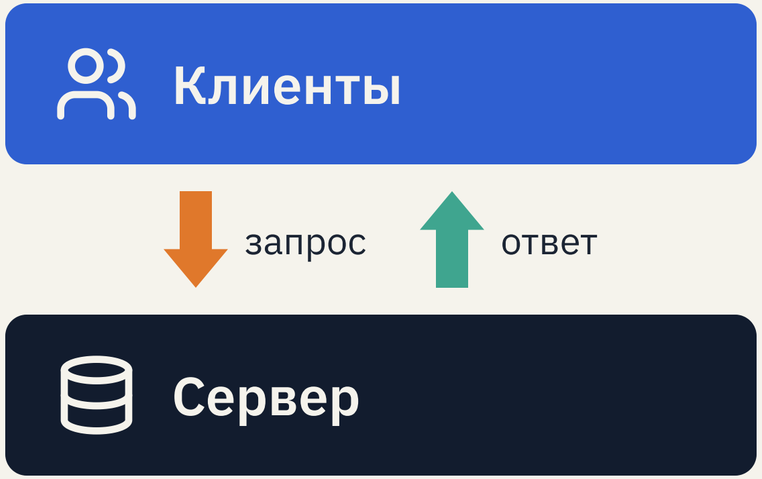
# Что такое архитектура клиент-сервер

**Клиент** отправляет запрос и показывает пользователю результат.

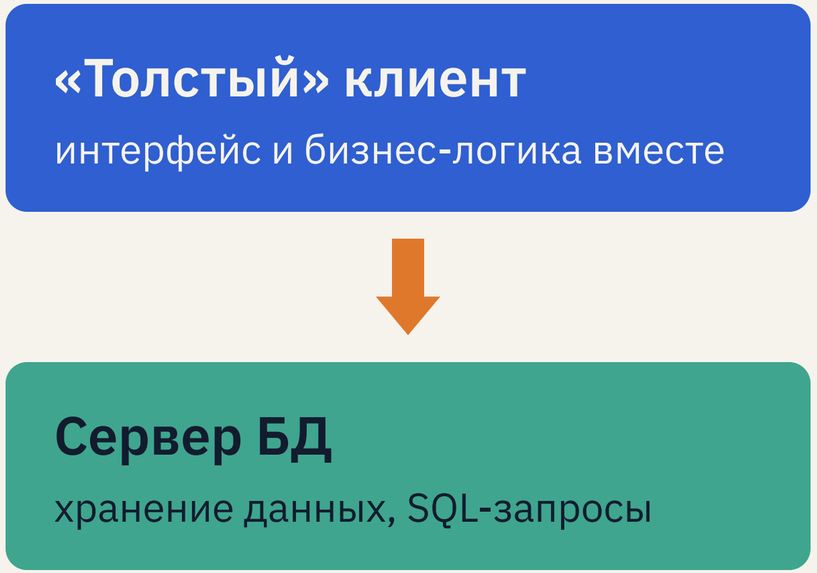
**Сервер** принимает запрос, обрабатывает его и возвращает ответ.

Обмен идет по сети по заранее известным протоколам, например HTTP и TCP/IP.

Один сервер обслуживает сразу много клиентов.



# Двухзвенная модель и ее пределы




## Где возникают проблемы

- Логику приходится обновлять на каждом рабочем месте.
- Клиент напрямую подключен к БД, а значит, выше риск для безопасности.
- Каждый клиент держит свое соединение, и база быстро перегружается.
- Систему трудно масштабировать и менять по частям.

# Три уровня архитектуры



Уровень 1

## Клиент

Представление: интерфейс и ввод данных



Уровень 2

## Сервер приложений

Логика: правила, проверки, транзакции



Уровень 3

## Сервер БД

Данные: хранение и выполнение запросов

Каждый уровень решает свою задачу и общается только с соседним.

# Уровень 1: клиент

## Уровень представления

- Показывает данные и принимает ввод пользователя.
- Чаще всего это «тонкий» клиент: браузер или мобильное приложение.
- Не содержит бизнес-логики и не знает, где лежат данные.
- Обновляется просто: достаточно обновить сервер.



# Уровень 2: сервер приложений

Здесь находится бизнес-логика: то, что делает систему именно этой системой.

**Бизнес-правила**

Расчеты, скидки, статусы заказов

**Проверка и доступ**

Аутентификация, права пользователей

**Транзакции**

Согласованное изменение данных

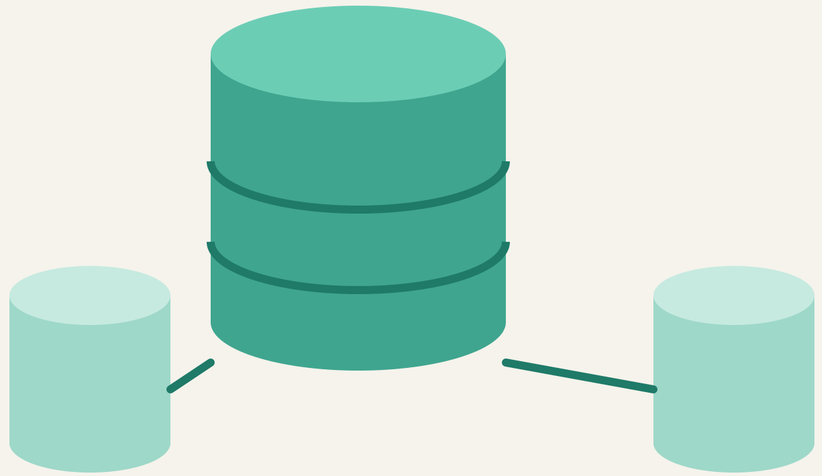
**Интеграция**

Обращение к другим сервисам и API

# Уровень 3: сервер базы данных

## Уровень данных

- Хранит данные и следит за их целостностью.
- Выполняет SQL-запросы от сервера приложений.
- Закрыт от клиентов: доступ только через средний уровень.
- Резервные копии и репликация защищают от потери данных.



# Путь запроса через систему

| № | Где | Что происходит |
|---|---|---|
| 1 | Клиент | Пользователь нажимает кнопку, уходит HTTP-запрос |
| 2 | Сервер приложений | Проверяет права и входные данные |
| 3 | Сервер приложений | Применяет бизнес-правила, формирует SQL |
| 4 | Сервер БД | Выполняет запрос и возвращает данные |
| 5 | Сервер приложений | Собирает ответ в виде JSON или HTML |
| 6 | Клиент | Показывает результат на экране |

# Двухзвенная и трехзвенная модели

| Критерий | Двухзвенная | Трехзвенная |
|---|---|---|
| Бизнес-логика | В клиенте | На сервере приложений |
| Доступ к БД | Напрямую | Через средний уровень |
| Обновление | На каждом ПК | Централизованно |
| Безопасность | Ниже | Выше |
| Масштабирование | Затруднено | Гибкое по уровням |
| Сложность и цена | Ниже | Выше |

# Преимущества

### Масштабируемость

Каждый уровень растет отдельно

### Безопасность

База скрыта от клиентов

### Сопровождение

Логика обновляется в одном месте

### Независимость

Уровни можно менять по отдельности

Дополнительно: легкие клиенты, повторное использование логики, удобное разделение работы между командами.

# Недостатки и ограничения

**Сложность.** Больше компонентов, настроек и развертываний.

**Задержки.** Появляется лишний сетевой переход между уровнями.

**Стоимость.** Нужны дополнительные серверы и администрирование.

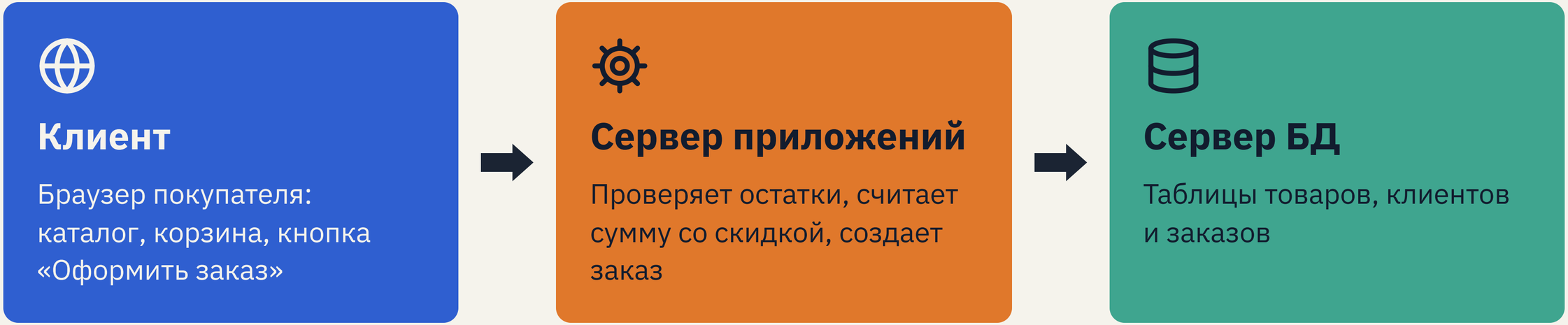
**Узкое место.** Один сервер приложений без резервирования становится точкой отказа.

# Технологии по уровням

| Уровень | Технологии | Примеры |
| --- | --- | --- |
| Клиент | HTML, CSS, JavaScript | React, Vue, Android |
| Сервер приложений | Java, C#, Python | Spring, ASP.NET, Django |
| Сервер БД | СУБД и язык SQL | PostgreSQL, MySQL |

Подробнее о многоуровневых архитектурах: Microsoft Learn, N-tier architecture style

# Пример: интернет-магазин




Покупатель видит только страницу магазина. Ни цены, ни правила скидок, ни таблицы базы ему напрямую недоступны.

# Масштабирование и N-звенность

- Серверов приложений можно запустить несколько: балансировщик распределяет запросы.
- Базу данных реплицируют, чтобы выдержать нагрузку и отказ узла.
- Уровней может быть больше трех: кэш, очереди, API (N-звенная архитектура).



# Выводы

**3** уровня: клиент, сервер приложений, сервер БД, и у каждого своя роль.



Логика и данные защищены от прямого доступа клиентов.



Систему проще масштабировать и сопровождать, но она сложнее и дороже.

# Источники и ссылки

Microsoft Learn: N-tier architecture style

IBM: What is three-tier architecture?

Wikipedia: Multitier architecture

Guru99: Двух-, трех- и N-уровневая архитектура

Prezi: материал курса по облачным презентациям